

Geo-Distributed Hybrid Retrieval-Augmented Generation: Placement Tradeoffs and Speculative Prefill Acceleration

M. Abdullah Ali*, M. Abdullah Aamir*

*Department of Data Science, FAST-NUCES, Islamabad, Pakistan
{m.abdullah.ali, m.abdullah.aamir}@fast.edu.pk

Abstract—Retrieval-Augmented Generation (RAG) systems are increasingly deployed across geographically distributed, heterogeneous nodes, yet the placement of their individual pipeline stages—sparse retrieval, dense retrieval, fusion, hydration, and generation—remains an open systems question. We present a cache-state-aware measurement study of stage-to-node placement for a *hybrid* RAG pipeline (BM25 + BGE-M3 dense retrieval with reciprocal-rank fusion, and a vLLM-served Qwen3.8-27B generator) on commodity hardware: an RTX 3090 generation host and a GTX 1660 Ti laptop gateway, connected over a WireGuard link shaped by *bidirectional* Linux *netem* across four WAN regimes (0/15/40/80 ms), with 50 seeded queries per arm, three repeats, and cold-prefix and warm cache states reported separately. Three findings reframe placement for this class of systems. First, TTFT is essentially *flat* across the swept RTT range (a measured sensitivity of 0.73 ms of TTFT per ms of RTT): an identifier-sized placement boundary makes the network regime a second-order concern. Second, KV-cache state, not the network, dominates latency variance—cold-prefix TTFT (≈ 3.5 s) is $4.4\times$ warm TTFT (≈ 0.82 s). Third, SPHP (Speculative Progressive Hydration & Prefill), which prefills the generator on a provisional sparse hint while the dense leg is still in flight, cuts cold TTFT by 17.4% (95% CI $[-20.1, -14.3]\%$) at every regime with a measured hit rate of $h = 0.84$, while warm-state savings are within noise; an answer-level comparison bounds the hit-path quality cost (token-F1 0.42 versus a 0.59 resampling-noise ceiling). We further derive an analytical placement cost model validated at the regime level (leave-one-tier-out MAPE 1.3–2.8%) and use it to show that the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space. All artifacts, including the measurement harness and scoring pipeline, are released.

Index Terms—Retrieval-Augmented Generation; hybrid search; edge-cloud systems; speculative inference; geo-distributed inference; placement optimization; KV-cache-aware measurement.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) has become the dominant architecture for grounding large language models (LLMs) in external knowledge [1]–[4]. A modern RAG pipeline is not a single monolith but a sequence of heterogeneous stages: a *sparse* lexical search (e.g., BM25 [5]), a *dense* semantic search over learned embeddings [6]–[8], a *fusion* step that merges the two ranked lists (e.g., reciprocal-rank fusion [9]), a *hydration* step that fetches the full text of the selected passages, and finally *generation*, in which an LLM [10], [11] is prefilled

with the retrieved context and autoregressively decodes an answer [12].

As these systems move from a single datacenter box to the edge, a fundamental *placement dilemma* emerges. The stages have wildly different resource profiles: dense retrieval and generation demand a GPU, sparse retrieval is CPU-friendly, and hydration is a cheap disk lookup. Large models require hardware most machines do not have, data-residency rules can force retrieval and generation onto different hosts, and cost pushes work toward whatever hardware exists. Yet, to the best of our knowledge, the literature cannot answer: *where should each stage of a hybrid RAG pipeline run, under which network conditions, and what exactly is lost when the topology degrades?* DNN partitioning is studied at the layer level [13]–[15] and LLM serving splits model *phases* across machines, but a measurement study of *which pipeline stage runs on which machine* in a multi-node hybrid RAG deployment is missing.

We close this gap with a deployed, instrumented two-node testbed and a cache-state-aware factorial campaign over placement variant $\times$ network regime. Our contributions are:

- **C1 (Measurement).** A cache-state-aware factorial measurement of hybrid RAG placement on commodity heterogeneous hardware (RTX 3090 + GTX 1660 Ti) over a WireGuard link shaped by *bidirectional* *netem* [16], [17] across four WAN regimes, with seeded disjoint query slices, warmup discard, three repeats, and percentile + bootstrap-CI reporting. We show that (i) TTFT is flat across 0–80 ms of RTT (slope 0.73 ms/ms), because the P2 boundary ships only identifier-sized payloads, and (ii) KV-cache state dominates TTFT variance: cold-prefix TTFT is $4.4\times$ warm TTFT (§IV).
- **C2 (Mechanism).** SPHP, a pipeline-level speculative protocol, measured for real: a provisional sparse hint is shipped to the generation node, a masked speculative prefill runs in parallel with the dense leg, and the result is reconciled against the fused context on arrival (set-overlap hit test). With a *measured* hit rate of $h = 0.84$, SPHP reduces cold-prefix TTFT by 17.4% at every regime; warm-state savings are within noise; and an answer-level comparison bounds the hit-path quality cost (§V).

- **C3 (Model and policy).** An analytical placement cost model whose macro predictions are validated at the decision granularity that matters—the network regime (leave-one-tier-out MAPE 1.3–2.8% for P2)—plus a crossover analysis showing that P2-SPHP is the optimal placement across the entire swept (RTT, bandwidth) space, and a live demonstration of the regime-level policy selecting placement (§VI–§VII).
- **C4 (Artifacts).** A fully open, reproducible testbed: the 8.84M-passage MS MARCO index [18], the benchmark harness, the scoring pipeline, the cost model, and all figures—every number in this paper regenerates from committed scripts and seeded run files (§VIII).

The remainder of the paper is organized as follows. §II reviews related work on RAG, hybrid retrieval, speculative inference, and edge–cloud systems. §III describes the two-node deployment and the placement taxonomy. §IV presents the empirical characterization. §V details the SPHP protocol. §VI develops the analytical placement cost model and the crossover analysis. §VII reports the evaluation, and §VIII–§IX discuss the results and conclude.

II. RELATED WORK

A. Retrieval-Augmented Generation

RAG couples a parametric generator with a non-parametric memory. Lewis *et al.* [1] introduced the generator–retriever formulation for knowledge-intensive tasks; REALM [2] and RETRO [3] showed that retrieving from trillions of tokens improves language modeling at scale. A recent survey [4] organizes the field into naive, advanced, and modular RAG, and active-retrieval variants such as FLARE [19] and the declarative compiler DSPy [20] adapt retrieval to the query. The closest systems-level study, Shen *et al.* [21], characterizes the latency, throughput, and memory trade-offs of a RAG stack (encode → retrieve → prefill → decode) and measures retrieval quality jointly with systems metrics—but on a *single host*, with no network leg and no placement variable. All of this work assumes a single, well-provisioned host; none addresses the placement of the retrieval and generation stages across a geo-distributed, heterogeneous fleet.

B. Hybrid Retrieval and Fusion

Hybrid search combines a lexical (sparse) and a semantic (dense) signal. BM25 [5] remains the lexical workhorse; dense passage retrieval [6] and sentence embeddings [7] provide the semantic leg, with multi-vector late-interaction models such as ColBERT [22] improving effectiveness at higher cost. Modern multi-functionality embedding models such as BGE-M3 [8] serve both dense and lexical objectives from one encoder. Reciprocal-rank fusion [9] is the standard parameter-light combiner. We use exactly this stack—Tantivy BM25, BGE-M3 dense search over Qdrant, and RRF—and measure how its two legs behave when split across machines.

C. Speculative LLM Inference

Speculative decoding accelerates autoregressive generation by drafting with a cheap model and verifying with a large one [23], [24]. Our SPHP mechanism is *orthogonal* to this line: rather than speculating over the *tokens* of a single model, we speculate over the *pipeline schedule*—we begin prefilling the generation node on a provisional (sparse) context before the dense leg has finished, then reconcile. This is a form of pipeline-level speculation that exploits the measured structure of the retrieval legs (§IV).

Two RAG-specific speculative systems are the closest neighbors, and both differ from SPHP on a distinct axis. *Speculative RAG* [25] decomposes RAG into drafting and verification: a small specialist LM drafts candidate answers from document subsets while a large generalist LM verifies and selects the best draft—a *model-level* draft/verify over generated answers, not a prefill on a provisional retrieval hint, and it reconciles by selecting the best draft rather than by a set-overlap test against the final fused result. *Omnia* [26] speculatively prefills the top reranked chunk while later reranking groups append sub-prefills—the closest to “prefill before the full context is ready”—but it speculates on a *reranked chunk* and has no reconciliation step. SPHP is distinct from both: it speculates on a *provisional sparse hint* (the BM25 leg’s top-*k*) and *reconciles* the speculative prefill against the *fused* (sparse+dense) context on arrival via a set-overlap hit test (§V).

The predict→act→reconcile pattern itself is long established in the systems literature, but never over provisional retrieval results. Informed prefetching [27] and predictive web prefetching [28] predict which *data objects* to fetch and simply evict/drop on a miss (no commit/abort/replay). Client speculation over replicated services [29] and OS-level application-specific speculation [30] predict a *remote result* and reconcile by value-equality or checkpoint/rollback. None speculates on a provisional *retrieval result* to prefill a generator, and none reconciles by set-overlap against a final fused result.

D. Edge–Cloud and Geo-Distributed Systems

Hybrid cloud–edge inference partitions a network between a device and a server [13]–[15], and datacenter-scale serving systems study phase and memory management [12], [31]. Secure point-to-point tunnels such as WireGuard [16] and kernel traffic control (`tc/netem`) [17] provide the transport and the controlled-WAN substrate we use. Energy and policy considerations for distributed deep learning [32] motivate placing expensive stages where hardware and power are cheapest.

Driving a placement decision from an analytical per-stage cost model is established practice in two adjacent settings, and our cost model (§VI) follows the same pattern while changing the object and the independent variable. In edge–cloud systems, Neurosurgeon [13] fits predictive models of per-layer latency and intermediate data size to choose a DNN split point between device and cloud, and Auto-Split [15] optimizes the split point coupled with quantization to minimize end-to-end latency. In LLM serving, Splitwise [33] uses an

analytical cost/power model of the two inference phases to place prefill and decode on different hardware, DistServe [34] co-optimizes per-phase resource allocation and parallelism and places the two phases by cluster bandwidth, AlpaServe [35] uses an analytical per-model latency/overhead model to place and parallelize a portfolio of models, and Janus [36] produces a placement decision as a function of link bandwidth and RTT for edge-adjacent prefill/decode disaggregation. These are complemented by queueing and bottom-up latency models that predict serving latency—the $M/G/1/K*PS$ web-server queue [37] and the bottom-up DNN-serving model of Clockwork [38]. All of these place *DNN layers* or *LLM phases* (with a KV-cache handoff) and treat hardware cost/power or cluster bandwidth as the placement variable. Our delta is threefold: we place *discrete RAG pipeline stages* (sparse/dense/ fusion/hydrate/generation), we use $RTT \times bandwidth$ as the independent variable to produce a regime-level placement phase map, and we drive a *live* placement policy validated against a measured campaign. Our work is the first to bring this placement lens to the RAG pipeline itself.

III. SYSTEM ARCHITECTURE & DEPLOYMENT

A. Testbed

We deploy a two-node hybrid RAG system over a WireGuard virtual private network. **Node A** (generation) is an RTX 3090 workstation (24 GB VRAM) serving a Qwen3.8-27B model quantized to W4A16 [11] under vLLM [12], together with a local SQLite store holding the full 8,841,823-passage MS MARCO corpus [18] for $O(1)$ hydration. The engine runs with speculative decoding enabled; because the model emits chain-of-thought deliberation by default, thinking is *suppressed per request* via chat-template kwargs so that TTFT measures the answer stream rather than reasoning tokens. **Node B** (edge gateway) is a laptop (Intel i7-10750H, GTX 1660 Ti) running the retrieval stack: a Tantivy BM25 index over the full 8.84M passages, a BGE-M3 dense encoder [8] with a 1024-dimensional HNSW index in Qdrant, and local reciprocal-rank fusion [9]. The two nodes are connected over WireGuard [16] (Node A at 10.8.0.1, Node B at 10.8.0.2) with gRPC streaming for the query, the fused context, and the returned token stream. Figure 1 shows the architecture and the warm steady-state per-stage timing annotations.

B. Placement Topologies

We define a taxonomy of five placement topologies by the location of the pipeline boundary (Figure 2): **P0** (colocated) runs every stage on Node A; **P1** places only the gateway on Node B with all compute on Node A; **P2** (our baseline) runs sparse, dense, and fusion on Node B and hydration, prefill, and decode on Node A, shipping only the top- k document identifiers across the link; **P2-SPHP** augments P2 with the speculative protocol of §V; and **P3** moves hydration to Node B as well, shipping the full text of the top passages. P2 is the natural split: it keeps the GPU-bound dense retrieval and generation on capable hardware while moving the cheap identifier payload—not the bulky corpus text—across the link.

C. Data Flow and Protocol

The client submits a query to the Node B gateway over HTTP; the gateway dispatches it to both retrieval legs concurrently over gRPC and, on completion, streams the fused top- k context (or the provisional sparse hint under SPHP) to Node A over the WireGuard tunnel. Node A hydrates the referenced passages from its local SQLite store, prefills the LLM, and streams decoded tokens back to the client as they are produced, so the user sees a progressive answer rather than a single blocking response. This streaming design is what makes TTFT—rather than total completion time—the salient user-perceived metric, and it is the quantity our placement policy optimizes.

D. Network Control

To study placement under controlled WAN conditions we shape the link with Linux `netem` [17] *bidirectionally*: a delaying `qdisc` on Node A’s egress interface applies $rtt/2$ of propagation to outbound packets, and an ingress `redirect (ifb)` applies $rtt/2$ to inbound packets, so that an end-to-end ping RTT increases by exactly the configured value. We validate the shaping before every measured window with a ping probe against a $\pm 30\%$ gate (e.g., the 15 ms regime measured 14.3 ms). We sweep four regimes—0, 15, 40, and 80 ms RTT—representing LAN, near-edge, and wide-area conditions, over the nominal 100 Mbps WireGuard link. Because both directions are shaped, every crossing of the A–B boundary (query forward, context return, hint forward, token stream back) experiences the configured delay—unlike an application-level sleep injected at one endpoint, which inflates only the metric it touches and reproduces no queuing or serialization behavior.

E. Measurement Protocol

All reported numbers come from a single scripted campaign: four regimes $\times$ two arms (P2 baseline, P2-SPHP) $\times$ 50 queries sampled with seed 42 from the MS MARCO dev set (6,980 queries with qrels), with *disjoint per-arm query slices* so that no query is seen by both arms and cross-arm cache warm-up cannot contaminate comparisons. Each arm executes five warm-up queries that are discarded, then three recorded repeats per query (1,200 measured records, zero failures). Repeat 0 of each arm slice is a *cold-prefix* reference (the engine retains no flush endpoint, so we report this state explicitly rather than pretending order-independence); repeats 1–2 are *warm*. We report means, medians, p95, and bootstrap 95% confidence intervals (2,000 resamples, seed 42) throughout. Retrieval quality is evaluated separately on 500 seeded dev queries against qrels, and answer-level SPHP effects on 200 seeded queries.

IV. EMPIRICAL CHARACTERIZATION

Table I summarizes the campaign for the P2 baseline arm, split by cache state.

Geo-Distributed Hybrid RAG: System Architecture

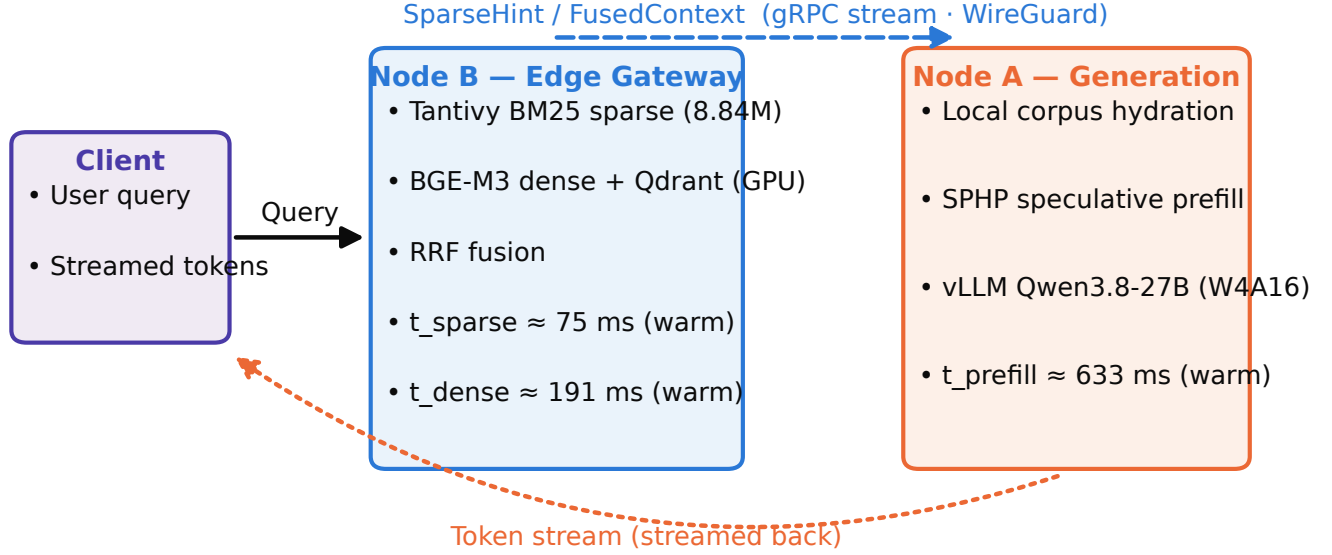


Fig. 1. System architecture. The client query enters the edge gateway (Node B), which runs sparse (Tantivy BM25) and dense (BGE-M3/Qdrant) retrieval in parallel and fuses them with RRF. The fused context (or a provisional sparse hint under SPHP) is streamed over the WireGuard link to the generation host (Node A), which hydrates the passages from local SQLite and runs the W4A16-quantized Qwen3.8-27B under vLLM. Tokens are streamed back to the client. Timings are warm steady-state means from the measurement campaign.

A. Leg Asymmetry and the TTFT Budget

The two retrieval legs are asymmetric, but far less dramatically than intuition suggests once the full deployment stack is measured: warm, the sparse (BM25) leg averages 66–80 ms and the dense (BGE-M3 + HNSW) leg 186–194 ms—a $2.6\times$ ratio at top- $k=10$ through the full gateway/gRPC path—while cold, the legs are 1,511–1,964 ms and 2,621–2,742 ms (a $1.6\times$ ratio; cold sparse is dominated by cold index and page caches, not by BM25 itself). The dominant term of the TTFT budget is neither leg: warm TTFT is ≈ 818 ms, of which retrieval contributes only ≈ 190 ms and the *prefill residual* (hydration + engine prefill + queueing) contributes ≈ 630 ms. The practical consequence inverts the naive reading of the warm asymmetry: because the two legs run in parallel and are within a small factor of each other, no placement can hide more than ≈ 100 ms of warm retrieval time behind the other—while the prefill, the largest single warm term, is exactly the component a speculative mechanism can attack (§V).

B. Cache State Dominates; the Network Barely Matters

The widest gap in Table I is vertical, not horizontal: cold-prefix TTFT is $4.4\times$ warm TTFT at every regime (≈ 3.5 s vs. ≈ 0.82 s). By contrast, TTFT is essentially flat *across* regimes: warm means move $817.9 \rightarrow 875.0$ ms from 0 ms to 80 ms of RTT, a measured sensitivity of 0.73 ms of TTFT per ms

of RTT (Figure 6); the +57 ms at the 80 ms tier is consistent with a single crossing of the link at identifier-sized payloads and is small against both the warm TTFT and the within-state spread. Cold means are flat within their wider confidence bands. Decode throughput is stable at ≈ 35 tok/s (per-cell medians 34.4–35.7) across all regimes and states, confirming that the link does not throttle the GPU-bound decode phase. The systems conclusion is stark: for identifier-sized placement boundaries, the network regime is a second-order term, and measurement protocols that do not separate cache states will attribute cold-start cost to whatever knob happens to be under study.

V. THE SPHP PROTOCOL

SPHP (Speculative Progressive Hydration & Prefill) converts the largest TTFT term—the prefill residual—into overlapped work. The key idea is to begin the generation-side prefill *before* the dense leg has finished, using a cheap provisional context, and to reconcile once the true fused context arrives.

A. Provisional Sparse Hint

As soon as the sparse leg completes at time T_{sparse} , Node B transmits a *provisional sparse hint*: the top-5 sparse document identifiers (a ≈ 104 -byte payload) over the link. This hint crosses the link in a fraction of a millisecond at identifier

Placement Topology Taxonomy (P0-P3)

	Client	Edge (Node B)	Cloud (Node A)
P0 • Colocated	• Query		• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P1 • All stages on A	• Query	• Gateway	• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P2 • Retrieval B, Gen A	• Query	• Sparse • Dense • Fusion	• Hydrate • Prefill • Decode
P2-SPHP • SPHP spec. prefill	• Query	• Sparse • Dense • Fusion	• Hydrate • Spec. Prefill • Decode
P3 • Retrieval + Hydrate B	• Query	• Sparse • Dense • Fusion • Hydrate	• Prefill • Decode

Fig. 2. Placement topology taxonomy (P0-P3). Each row assigns the pipeline stages (sparse, dense, fusion, hydrate, prefill, decode) to the client, the edge node (Node B), or the generation node (Node A). P2-SPHP is P2 with a speculative prefill that overlaps the dense leg.

payloads, and it is a legitimate, if incomplete, approximation of the final fused ranking.

B. Speculative Prefill with Masking

On receiving the hint, Node A immediately hydrates the five provisional passages from local SQLite and begins a *speculative prefill* of the LLM on this provisional context. Crucially, the prefill is *masked*: its output is held in a scratch buffer and is *not* emitted to the client. The speculative prefill runs in parallel with the still-in-flight dense leg (Figure 3).

C. Reconciliation

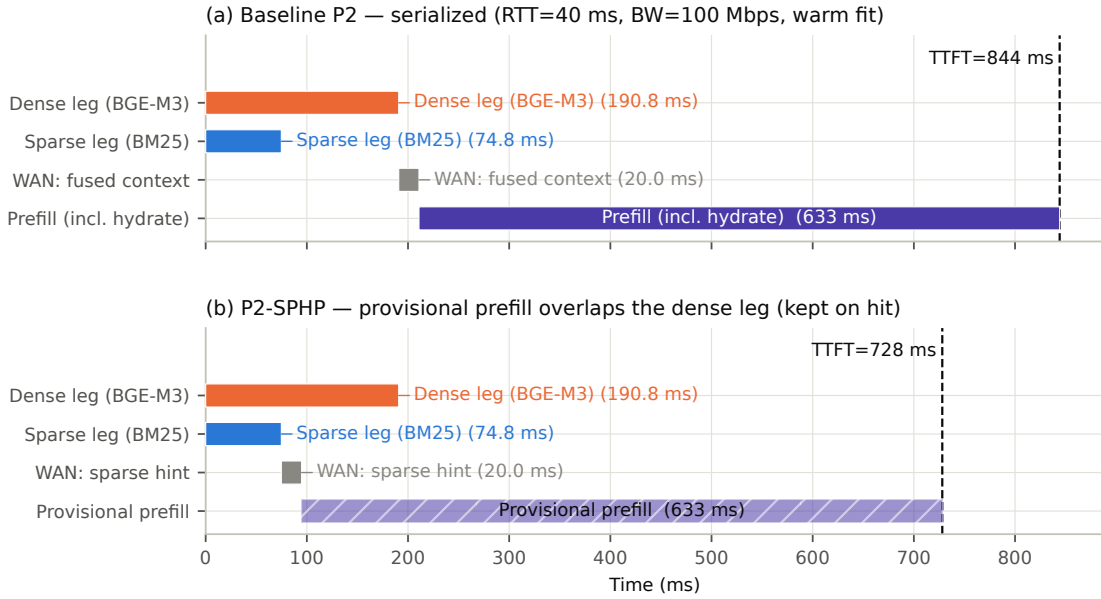
The dense leg completes at T_{dense} and the fused context (top-10 identifiers) arrives at Node A at $t_{\text{fused}} = T_{\text{dense}} +$

$T_{\text{fusion}} + T_{\text{WAN}}$. SPHP then *reconciles*. The implemented test is a set-overlap rule: the speculation is declared a *hit* when the provisional hint covers at least half the hint set, i.e., $|\mathcal{H} \cap \mathcal{F}| \geq 0.5 |\mathcal{H}|$, where $\mathcal{H}$ is the top-5 sparse hint and $\mathcal{F}$ the fused top-10 (Algorithm 1). On a hit, the speculative prefill is kept and the first token is emitted immediately—the provisional KV prefix is reused, so the prefill is removed from the critical path. On a miss, the speculative buffer is discarded and a standard P2 prefill runs on the true context; the wasted provisional prefill is a *measured* penalty T_{miss} (193.6 ms warm, 2,697.8 ms cold in our campaign). With the *measured* hit rate $h = 0.84$ (95% CI [0.80, 0.87]; per-regime 0.82–0.90), the expected TTFT is the hit-rate-weighted combination of the two cases.

TABLE I

P2 BASELINE PER-STAGE SUMMARY BY REGIME AND CACHE STATE (50 QUERIES PER CELL; COLD = FIRST REPEAT OF AN ARM SLICE, WARM = REPEATS 1–2). SPARSE AND DENSE LEGS EXECUTE IN PARALLEL; TTFT IS THE FIRST-TOKEN LATENCY AND SUBSUMES THE PREFILL RESIDUAL. DECODE THROUGHPUT IS THE PER-CELL MEDIAN.

Regime	State	Sparse (ms)	Dense (ms)	TTFT (ms)			Total (ms)	TPS (tok/s)
				mean	p50	p95		
0 ms	cold	1964.2	2742.2	3633.6	3674.1	4251.9	4843.3	34.8
0 ms	warm	75.9	193.5	817.9	819.6	1044.8	1701.2	34.4
15 ms	cold	1511.0	2697.2	3569.7	3607.2	4262.4	5586.4	35.4
15 ms	warm	65.7	191.7	815.8	817.7	1012.5	1999.3	35.2
40 ms	cold	1649.4	2663.3	3555.7	3522.7	4298.1	4465.4	35.7
40 ms	warm	79.8	186.3	817.1	806.3	1047.7	1638.4	35.5
80 ms	cold	1720.6	2620.6	3544.6	3503.8	4573.8	4729.5	35.3
80 ms	warm	77.8	191.7	875.0	851.5	1109.5	2039.3	35.7



Measured TTFT delta vs. baseline ($h = 0.84$ measured): cold -17.4% [95% CI -20.1, -14.3%], warm -2.0% [CI -5.1, +1.1%]

Fig. 3. SPHP timeline at RTT=40 ms, BW=100 Mbps, driven by the warm campaign-fitted stage components. (a) Baseline P2 is fully serialized: retrieval, fusion, transfer of the fused context, and prefill. (b) Under SPHP, the provisional sparse hint triggers a masked speculative prefill that overlaps the dense leg; on a hit, the first token is emitted at the later of the fused-context arrival and the speculative-prefill completion. The footnote reports the *measured* TTFT deltas with bootstrap CIs.

D. Measured Effect and Answer-Level Cost

Across the campaign, SPHP reduces cold-prefix TTFT by 17.4% (95% CI [-20.1, -14.3]%) at every regime (e.g., 2,948 ms vs. 3,634 ms at 0 ms RTT), while warm-state means differ by -2.0% (95% CI [-5.1, +1.1]%)—within noise. The mechanism is visible in the per-record instrumentation: on hits the wasted prefill is 0 ms (the provisional KV prefix is kept), while misses pay T_{miss} ; at $h = 0.84$ the expected value is favorable, and decisively so in the cold state where the baseline prefill is largest.

Because a hit emits the answer from the *sparse-only* provisional context, we quantify the answer-level effect directly:

for 200 seeded dev queries we run the baseline and SPHP paths and compare their generated answers. As a control, the miss arm re-generates from the *identical* fused context and measures pure decode stochasticity: its answers agree with the baseline at token-F1 0.586 (EM 0.121)—the sampling-noise ceiling. The hit-path answers agree at token-F1 0.422 (EM 0.042). We therefore do *not* claim answer parity: hit-path answers shift measurably beyond sampling noise, because the provisional context omits the dense contributions. The cost is bounded (comparable in magnitude to decode stochasticity), the retrieved *ranking* pipeline is untouched, and the miss path is bit-identical to P2 by construction.

Algorithm 1 SPHP: Speculative Progressive Hydration & Prefill

Require: query q ; sparse leg T_{sparse} ; dense leg T_{dense} ; WAN delay T_{WAN} ; hit threshold $\theta = 0.5$

- 1: **Node B:** run sparse and dense legs in parallel
- 2: **Node B:** on sparse completion, send provisional hint $\mathcal{H}$ (top-5 sparse ids) to Node A
- 3: **Node A:** hydrate $\mathcal{H}$; start *masked* speculative prefill $\hat{P}$
- 4: **Node B:** on dense completion, fuse (RRF) and send $\mathcal{F}$ (top-10 ids) to Node A
- 5: **Node A:** on $\mathcal{F}$ arrival, reconcile
- 6: **if** $|\mathcal{H} \cap \mathcal{F}| \geq \theta |\mathcal{H}|$ (*hit*) **then**
- 7: keep $\hat{P}$; emit first token now
- 8: **else**(*miss*)
- 9: discard $\hat{P}$; run standard prefill on $\mathcal{F}$; pay T_{miss}
- 10: **end if**
- 11: stream decoded tokens to the client

VI. ANALYTICAL PLACEMENT COST MODEL

We derive a closed-form cost model that predicts TTFT for each topology as a function of the network regime, fit its parameters per cache state to the campaign, and validate it at the regime level.

A. WAN Transfer Term

A one-way WAN transfer of a p -byte payload over a link with round-trip time rtt and bandwidth bw (Mbps) is

$$T_{\text{WAN}}(p, \text{rtt}, \text{bw}) = \frac{\text{rtt}}{2} + \frac{8p}{\text{bw} \times 10^6} \times 10^3 \text{ ms.} \quad (1)$$

The first term is propagation (half the RTT); the second is serialization. Payload sizes are $p_{\text{id}} = 64 + 10 \times 8 = 144$ B (fused identifiers), $p_{\text{hint}} = 64 + 5 \times 8 = 104$ B (sparse hint), and $p_{\text{text}} = 64 + 5 \times 1200 = 6064$ B (hydrated passages).

B. Topology Costs

Let the retrieval phase be $T_{\text{ret}} = \max(T_{\text{sparse}}, T_{\text{dense}})$ (parallel legs) and let hydration cost T_{hydrate} for the top passages. The four topologies are:

$$\text{P0: } T_{\text{TTFT}}^{(0)} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{prefill}}, \quad (2)$$

$$\text{P2: } T_{\text{TTFT}}^{(2)} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{WAN}}(p_{\text{id}}) + T_{\text{prefill}}, \quad (3)$$

$$\text{P3: } T_{\text{TTFT}}^{(3)} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{WAN}}(p_{\text{text}}) + T_{\text{prefill}}, \quad (4)$$

where P0 is colocated, P2 ships only identifiers, and P3 ships full text. For P2-SPHP, define the speculative-prefill start $t_s = T_{\text{sparse}} + T_{\text{WAN}}(p_{\text{hint}}) + T_{\text{hydrate}}$, its completion $t_p = t_s + T_{\text{prefill}}$, and the fused-context arrival $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}(p_{\text{id}})$. On a hit the first token is emitted at $\max(t_f, t_p)$; on a miss it falls back to P2 plus the measured wasted-prefill penalty T_{miss} . The expected TTFT is

$$T_{\text{TTFT}}^{(2, \text{SPHP})} = h \max(t_f, t_p) + (1 - h)(T_{\text{TTFT}}^{(2)} + T_{\text{miss}}). \quad (5)$$

The total end-to-end latency adds decode and token-streaming time:

$$T_{\text{e2e}} = T_{\text{TTFT}} + \frac{N_{\text{tok}}}{R_{\text{decode}}} \times 10^3 + T_{\text{WAN}}(N_{\text{tok}} \times 4). \quad (6)$$

C. Per-State Fitting and Regime-Level Validation

Every parameter is fitted from the campaign separately per cache state, because the components differ by an order of magnitude between them: warm $T_{\text{sparse}}=74.8$ ms, $T_{\text{dense}}=190.8$ ms, $T_{\text{prefill}}=633.3$ ms (the residual after measured retrieval and fusion); cold 1,711.3, 2,680.9, and 850.7 ms respectively; $R_{\text{decode}}=35.2$ tok/s; $h=0.84$ and T_{miss} measured as above. *Why regime-level validation:* query-level TTFT variance is dominated by prompt content rather than placement, so a population-mean predictor cannot and should not be scored per query (a per-query protocol yields a negative R^2 for any mean predictor, ours included). The decision a placement policy actually makes is *per regime*, so we validate exactly that: leave-one-tier-out, fitting all parameters on three RTT tiers and predicting the held-out tier's mean. The macro P2 model achieves **1.28%** (cold) and **2.76%** (warm) held-out regime MAPE; the SPHP expected-value model 10.6% and 5.8%. As a form-error diagnostic, the component-level check—which substitutes the observed per-record retrieval time and scores only the network, hydration, and prefill terms—yields 3.0% (cold) and 11.3% (warm) MAPE, and the SPHP *hit-branch* form check 7.4% (cold) / 14.8% (warm) (Figure 4).

D. Crossover Analysis

To evaluate where physical pipeline boundaries matter, we sweep the fitted model over $\text{RTT} \in [0, 200]$ ms and bandwidth $\in [10, 1000]$ Mbps for both cache states (Figure 5). The result is uniform: the SPHP-augmented split placement (P2-SPHP) is optimal in *every* cell of the sweep, with a modeled warm-state advantage of 7–8% that is essentially independent of both RTT and bandwidth, because the hint and fused-context payloads (104 and 144 B) serialize in well under a millisecond at any practical bandwidth. P3 is indistinguishable from P2 for TTFT at ≥ 10 Mbps (within 5 ms—six kilobytes of hydrated text barely matters); its serialization penalty only becomes visible under bandwidth starvation (≈ 96 ms at 0.5 Mbps, Figure 5(b)). We note honestly where the closed form diverges from measurement: the model's expected-value form *underestimates* the measured cold-state SPHP benefit (7.8% modeled vs. 17.4% measured) because a hit also preserves the provisional KV prefix—a credit the closed form does not encode—while the measured warm-state advantage is within noise where the model predicts a small positive effect. The model is therefore a conservative lower bound on the cold-start benefit and an optimistic upper bound on the warm-state one.

VII. EVALUATION

A. TTFT versus RTT: Measured

Figure 6 plots *measured* TTFT against the four netem regimes with bootstrap 95% CIs, for both arms and both cache states. In the cold panel the SPHP curve sits strictly below the baseline at every regime (the -17.4% mean delta); in the warm panel the curves overlap within their confidence bands. In neither panel does TTFT rise meaningfully with RTT: the identifier-sized boundary makes the network a second-order

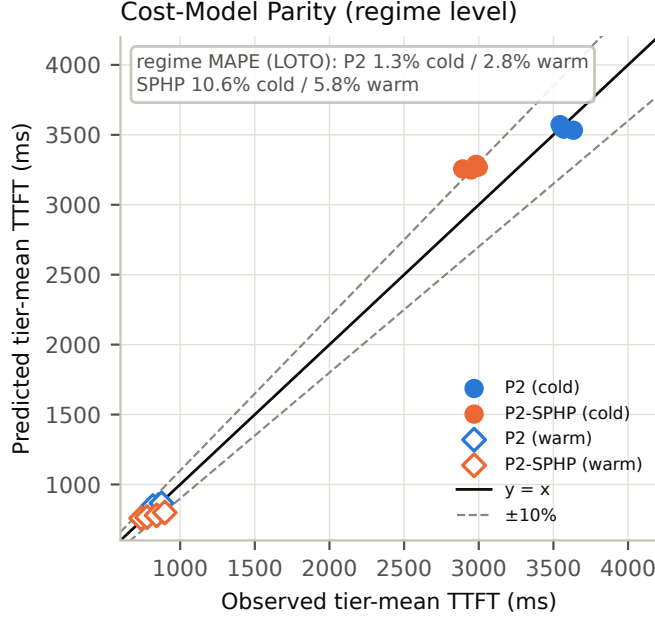


Fig. 4. Regime-level cost-model parity: predicted vs. observed per-tier mean TTFT for P2 and P2-SPHP, cold (filled) and warm (open), against the perfect-prediction diagonal with $\pm 10\%$ bands. Annotations report leave-one-tier-out regime MAPE.

term, and the SPHP benefit comes from overlapping the prefill with retrieval, not from any property of the link.

B. TTFT Composition

Figure 7 decomposes TTFT into the retrieval barrier ($\max(T_{\text{sparse}}, T_{\text{dense}})$), fusion, and the prefill residual, per regime and cache state. The cold panels show the retrieval barrier dominating (≈ 2.6 – 2.7 s, cold caches); the warm panels show the prefill residual dominating (≈ 620 – 680 ms of ≈ 820 ms). This is the quantitative case for SPHP: the component it removes from the critical path is the largest warm term and a large cold term.

C. Retrieval Quality

A placement or speculative mechanism is only acceptable if it does not degrade retrieval quality. Figure 8 reports retrieval metrics computed live on the deployed Node B gateway over 500 seeded MS MARCO dev queries (seed 42; top- $k=100$, RRF $k=60$). Dense retrieval (BGE-M3) is the precision champion (MRR@10 0.321, nDCG@10 0.377), while Hybrid (RRF) is the coverage champion: Recall@100 0.851 (vs. 0.848 dense, 0.669 sparse) and Recall@1000 **0.895** (vs. 0.848 dense—the sparse leg is capped at $k=100$ and cannot extend). Sparse BM25 is weakest on every metric. These numbers ground the hybrid design: RRF buys a small top-rank precision discount for a material recall margin.

D. Throughput

Decode throughput is stable at ≈ 35 tok/s (per-cell medians 34.4–35.7 tok/s; Table I) across all regimes and cache states, confirming that the link does not throttle the GPU-bound decode phase and that SPHP’s speculative prefill does not contend with

decode for GPU resources. Warm end-to-end totals of 1.6–2.0 s against TTFT of ≈ 0.82 s reflect the decode phase’s share of the budget, which placement—and SPHP—leave intact.

E. Live Policy Demonstration

The cost model is not only retrospective: at deployment time it reads the fitted parameters, predicts P2 and P2-SPHP TTFT for the *current* network regime, and selects the argmin placement. We demonstrate this live: under each netem regime the policy selects a placement, executes live queries, and logs the predicted versus realized TTFT. In the demonstration the policy selects P2-SPHP at *every* regime, consistent with the offline crossover analysis. Each regime serves five fresh seed-42 queries three times back-to-back (repeat 0 is the cold-prefix reference; repeats 1–2 are warm), after two unrecorded warmups, at $k=10$; selection uses the steady-state (warm) fit.

Table II reports the outcome. In the warm state the realized mean TTFT of the selected placement tracks its prediction to within 4.5–8.6% across all four regimes—inside the 15% tolerance RQ 3 asks of the policy, and comparable to the regime-level validation errors of the fit itself. In the cold state the model *overestimates* absolute TTFT by 25–36%: fresh contexts in the demo realized 2.1–2.4 s against cold predictions of ≈ 3.3 s. The error is conservative and leaves the argmin unchanged (the cold fit also selects P2-SPHP), but its direction is informative: the campaign’s cold reference carries the prefix-cache pressure of a sustained 1,200-request run, whereas the demo’s lighter history evicts less, so realized cold cost lands below the fitted worst case. The live run therefore re-confirms the paper’s central operational finding from the policy side: a single-shot deployment serving only fresh contexts runs at $\approx 3\times$

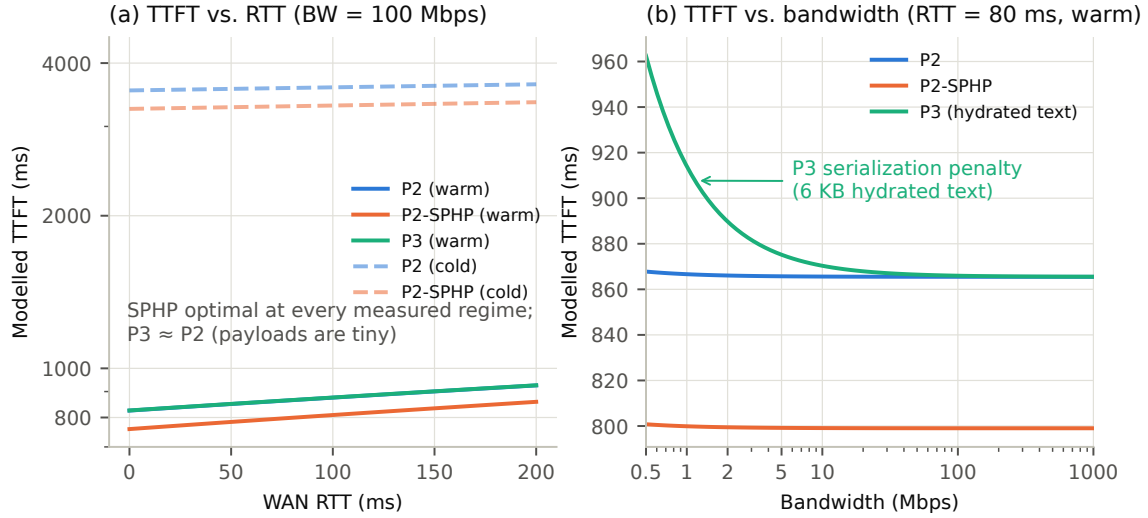


Fig. 5. Placement scenario analysis from the fitted model. (a) Modelled TTFT vs. RTT at 100 Mbps (log scale), warm solid and cold dashed: the topology ordering is invariant across the RTT range. (b) Modelled TTFT vs. bandwidth at 80 ms RTT: P3’s hydrated-text serialization penalty appears only under bandwidth starvation; P2 and P2-SPHP are flat.

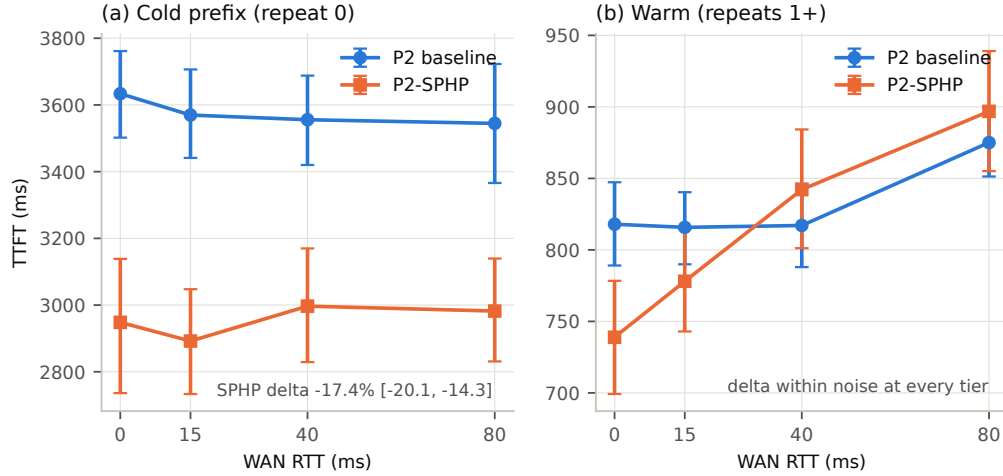


Fig. 6. Measured TTFT vs. WAN RTT (50 seeded queries per cell, 3 repeats; error bars are bootstrap 95% CIs of the mean). (a) Cold prefix: SPHP is below baseline at every regime. (b) Warm: the arms overlap within noise. TTFT is flat in RTT in both states.

steady-state TTFT regardless of placement, and a placement policy must condition on cache state—which the fitted model carries as explicit per-state parameters.

F. Summary of Gains

SPHP reduces cold-prefix TTFT by 17.4% (95% CI $[-20.1, -14.3]\%$) consistently across all four regimes, at a measured hit rate of 0.84; warm TTFT is unchanged within noise. The saving is insensitive to both RTT and bandwidth because the extra protocol message is 104 bytes and the speculation targets the prefill, not the network. The mechanism costs one small extra message, a masked prefill that is discarded only on a miss ($T_{\text{miss}}=194$ ms warm, 2,698 ms cold, measured), and a bounded answer-level shift on the hit path (§V).

TABLE II
LIVE POLICY DEMONSTRATION: PREDICTED VS. REALIZED TTFT OF THE SELECTED PLACEMENT (P2-SPHP AT EVERY REGIME), WARM CACHE STATE. $n=5$ QUERIES $\times$ 2 WARM REPEATS PER REGIME; ϵ IS THE ABSOLUTE PREDICTION ERROR.

RTT	Predicted (ms)	Realized (ms)	ϵ
0 ms	759.0	716.6	5.6%
15 ms	766.5	731.7	4.5%
40 ms	779.0	827.7	6.2%
80 ms	799.0	730.2	8.6%

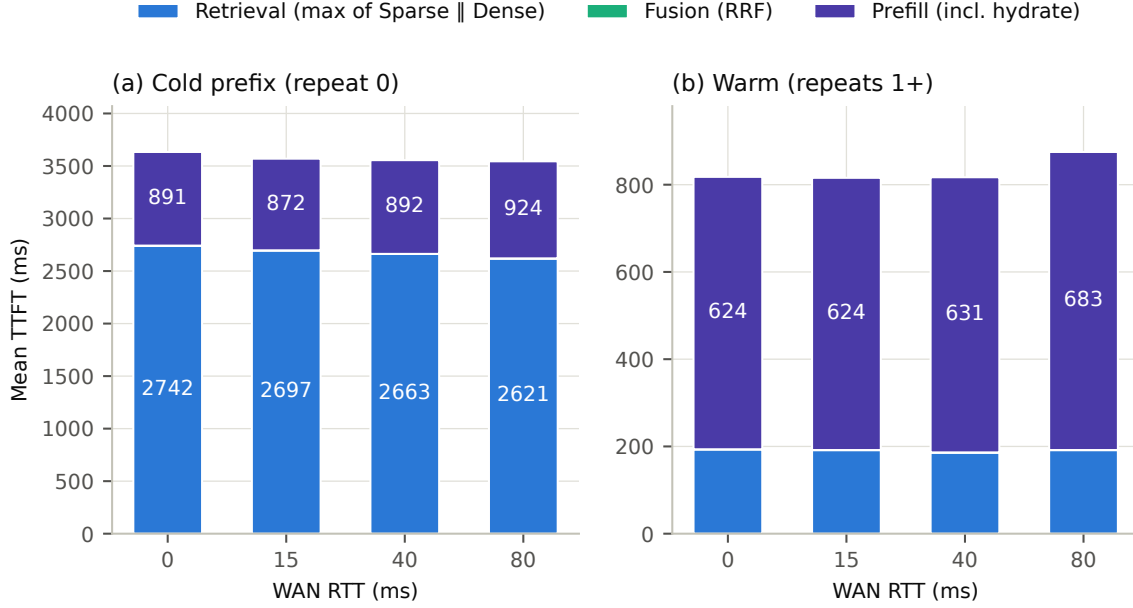


Fig. 7. Mean TTFT composition per regime and cache state (P2 baseline): retrieval barrier (max of the parallel sparse and dense legs), RRF fusion (visually negligible at ≈ 0.02 ms), and the prefill residual (hydration + engine prefill + queueing).

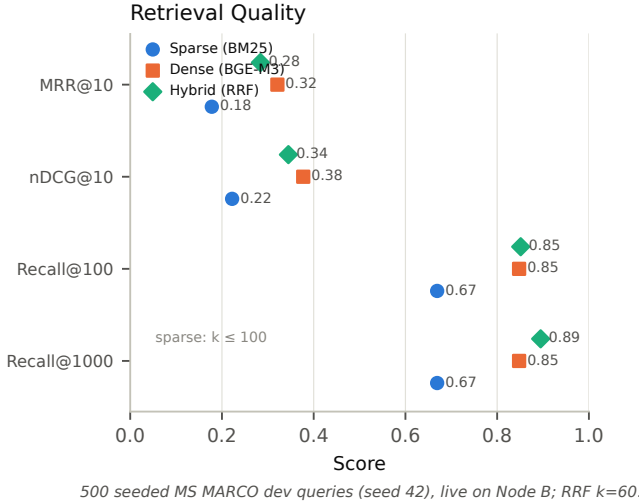


Fig. 8. Live retrieval quality on 500 seeded MS MARCO dev queries against the deployed Node B gateway (top- $k=100$, RRF $k=60$). Dense optimizes top-rank metrics; Hybrid (RRF) maximizes recall, including Recall@1000.

VIII. DISCUSSION

Cache state and prefill—not RTT—are the design drivers.

Our central empirical results invert the folklore that WAN placement is mainly about network delay: with identifier-sized boundaries, TTFT is flat across 0–80 ms while cache state swings TTFT by $4.4\times$. Placement decisions for hybrid RAG should therefore be made from a cache-state-aware measurement matrix, and deployments should spend their engineering budget on keeping prefixes warm (or making cold

starts cheap, which is precisely SPHP’s shape) rather than on micro-optimizing the link.

A simple, robust policy. The crossover analysis’s uniformity—P2-SPHP optimal in every swept cell—is a practically important result: a deployer need not re-solve placement per network regime. A single SPHP-augmented split placement is a safe default across the (RTT, bandwidth) space we studied, and the regime-validated cost model (§VI) provides the analytical basis—and the live policy hook—for extending the policy to new regimes.

Threats to validity. *Hardware confound:* the dense leg runs on a laptop-class GTX 1660 Ti, so absolute leg times (and the $2.6\times$ warm asymmetry) are hardware-specific; the ratios we report are qualified by k and by the measurement path, and the qualitative conclusions (prefill dominance, cache dominance, RTT flatness) are robust to plausible hardware changes. *Emulation fidelity:* *netem* shapes delay bidirectionally but with zero loss, no jitter, no reordering, and no route asymmetry; genuine WAN validation is future work (§VIII-A). *Single dataset and generator:* results are MS MARCO-specific (verbose web queries) and tied to one W4A16-quantized 27B generator with speculative decoding enabled; the decode rate (≈ 35 tok/s) and prefill residual are model-specific. SPHP’s hit rate is a property of the sparse–dense overlap distribution and may differ on other corpora or embedders. *No concurrency:* all measurements are single-query streams; queueing under concurrent load is out of scope. *Cold-state semantics:* repeat 0 is a cold-prefix reference under a resident engine, not a cold process start; we report it explicitly rather than conflating the two.

Relation to layer- and phase-level partitioning. Neurosur-

geon [13] and its successors partition a single neural network across a device and a server at the *layer* boundary, and LLM serving systems such as Splitwise partition a model’s *phases* (prefill vs. decode) across machines. Our setting is different in two ways. First, the unit of placement is a *pipeline stage* of a RAG system (sparse, dense, fusion, hydration, generation), not a layer or a phase of one model, so the stages have heterogeneous and independently scalable resource profiles. Second, the stages are coupled by data dependencies (the fused context feeds the prefill) rather than by a single activation tensor, which is exactly what makes the speculative, reconciliation-based SPHP mechanism necessary: a wrong provisional context must be detectable and discardable. Extending speculation across more than two stages multiplies the reconciliation surface: each additional provisional context turns the hit test into a set-membership check over a larger provisional family, and misses compound—a direction we discuss below.

Reproducibility. We release the full 8.84M-passage index, the benchmark harness (campaign driver, netem control, seeded query sampling), the scoring pipeline (retrieval-quality and answer-level comparison scripts), the cost model and its refit, and all figure-generating code and run files as open artifacts (C4). Every number in this paper regenerates from committed scripts with the seeds disclosed above.

A. Future Work and Three-Node Validation Roadmap

Three primary avenues build directly upon this foundation. First, a *three-node physical WAN campaign*: a third node enrolls as a *sparse-retrieval and user-facing tier*—serving the client directly while offloading the Tantivy index—so that the sparse hint originates one further hop away and the placement taxonomy gains a genuinely three-way split; executing it over a public cross-site route will validate our netem predictions under genuine Internet jitter, loss, and reordering. Second, quantifying *graceful degradation*: measuring latency and quality impacts when an edge node experiences transient disconnects and pipeline stages consolidate. Third, generalizing SPHP beyond sparse–dense pairs to multi-stage speculative prefetching across distributed multi-hop topologies, where the reconciliation test becomes a set-membership check over a larger provisional set.

IX. CONCLUSION

We presented a cache-state-aware measurement study of stage-to-node placement for a hybrid RAG pipeline on commodity heterogeneous hardware over a controlled WAN. The measurements reorder the design space: TTFT is flat across 0–80 ms of RTT (0.73 ms/ms), cache state dominates variance (4.4×), and the prefill residual is the largest warm TTFT term. SPHP exploits exactly that structure—a provisional sparse hint, a masked speculative prefill overlapping the dense leg, and a set-overlap reconciliation—cutting measured cold-prefix TTFT by 17.4% at every regime at a measured hit rate of 0.84, with the answer-level cost bounded and disclosed. An analytical placement cost model, validated at the regime level (1.3–2.8% leave-one-tier-out MAPE), shows the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth)

space. These results give practitioners a measured, model-based answer to where each stage of a hybrid RAG pipeline should run, and a reproducible testbed to extend the study to larger fleets and genuine wide-area conditions.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9781–9793.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *International Conference on Machine Learning (ICML)*, 2020, pp. 3979–3988.
- [3] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, A. Lucier, F. Hutter *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.
- [4] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” 2024.
- [5] S. Robertson and S. Walker, “Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval,” in *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1994, pp. 232–241.
- [6] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [7] N. Reimers and T. Gurewitz, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Jiang, “M3-embeddings: Multi-lingual, multi-functionality, multi-granularity text embeddings through symmetric distillation,” 2024.
- [9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 135–142.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [11] Qwen Team, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [13] W. Kang *et al.*, “Neurosurgeon: Hybrid cloud-edge inference services,” in *Proceedings of the 2017 ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 433–446.
- [14] S. Teerapittayanon, B. McDanel, and H.-T. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 1–11.
- [15] A. Banitalebi-Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, and Y. Zhang, “Auto-Split: A general framework of collaborative edge-cloud AI,” in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD)*, 2021, pp. 2543–2553.
- [16] J. A. Donenfeld, “WireGuard: Next generation kernel network tunnel,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.
- [17] “netem(8): Network emulator (tc qdisc),” *iproute2 manual page*, 2024, <https://man7.org/linux/man-pages/man8/tc-netem.8.html>.
- [18] Y. Bajaj, N. Craswell, M. Mitra, and M. Chen, “MS MARCO: A human-annotated open-domain dataset for machine reading comprehension,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 663–668.

- [19] Z. Jiang *et al.*, “Active retrieval augmented generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7963–7979.
- [20] O. Khattab *et al.*, “DSPy: Compiling declarative language model calls into self-improving pipelines,” 2023.
- [21] Y. Shen, M. Umar, S. Maeng, J. Suh, and I. Gupta, “Towards understanding systems trade-offs in RAG model inference,” 2024.
- [22] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–47.
- [23] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning (ICML)*, 2023, pp. 2992–3003.
- [24] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J.-J. Cappe, “Accelerating large language model decoding with speculative sampling,” 2023.
- [25] Y. Wang *et al.*, “Speculative RAG: Enhancing retrieval-augmented generation through drafting,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [26] Omnia *et al.*, “Efficient RAG serving through speculative scheduling,” in *Proceedings of the ACM International Conference for High Performance Computing, Networking, Storage and Analysis (HPDC)*, 2025, doi 10.1145/3806645.3807814.
- [27] R. H. Patterson, G. A. Gibson, E. Ginting, D. Stodolsky, and J. Zelenka, “Informed prefetching and caching,” in *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, 1995, pp. 79–95.
- [28] V. N. Padmanabhan and J. C. Mogul, “Using predictive prefetching to improve world wide web latency,” in *Proceedings of the ACM Conference on Communications Architectures, Protocols and Applications (SIGCOMM)*, 1996, pp. 22–36.
- [29] B. Wester, P. M. Chen, J. Cowling, J. Flinn, E. B. Nightingale, and B. Liskov, “Tolerating latency in replicated state machines through client speculation,” in *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2009.
- [30] B. Wester, P. M. Chen, and J. Flinn, “Operating system support for application-specific speculation,” in *Proceedings of the 6th European Conference on Computer Systems (EuroSys)*, 2011.
- [31] L. A. Barroso and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Morgan & Claypool Publishers, 2009.
- [32] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.
- [33] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.
- [34] Y. Zhong, Z. Liu, J. Gu, X. Jin, J. Chen, M. Park, Y. Liu, J. Pang, Y. Sheng, S. Han *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [35] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, “Alpaserve: Statistical multiplexing with model parallelism for deep learning serving,” in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 663–679.
- [36] Aruna, Kaliraj, Sudha, Sureka *et al.*, “Janus: Joint prefill/decode disaggregation with KV-cache-aware multi-cloud routing for edge-adjacent LLM serving,” *EAI Endorsed Transactions on the Internet of Things*, 2025, doi 10.4108/eetiot.13349.
- [37] J. Cao, M. Andersson, C. Nyberg, and M. Kihl, “Web server performance modeling using an M/G/1/K*PS queue,” *IEEE/ACM Transactions on Networking*, 2000.
- [38] A. Gujarati, R. Karimi *et al.*, “Serving DNNs like clockwork: Performance predictability from the bottom up,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.